
DTU

62550 Brugerinteraktion og udvikling på mobile enheder

UI programmering

Oversigt

- ❑ Android studio
- ❑ Reaktiv programmering
- ❑ Jetpack Compose
- ❑ Introduktion til Applikationer
- ❑ Gradle

Imperativ Programming

Imperativ: -bøjningskategori (modus) af et verbum der udtrykker ordre eller opfordring

https://ordnet.dk/ddo/ordbog?query=imperativ

Imperativ Programming

- Program er indelt i instruktioner (odre)
- Fokuseret på **Hvordan**
- Ændre state via instruktioner
- Instruktionerne følges sekventielt

Deklarativ Programming

Deklarativ: -oplysende; forklarende;

https://ordnet.dk/ods/ordbog?query=deklarativ

Deklarativ Programming

Deklarativ: -oplysende; forklarende;

(Inden for software, omfatter det programmering som ikke er imperativ)

https://ordnet.dk/ods/ordbog?query=deklarativ

Deklarativ **Programming**

- Program er indelt i logiske udtryk
- Fokuseret **Hvad**
- Beskriver det ønskede slutresultat

Imperativ eller Deklarativ programmering?

```
// Imperativt eksempel

val numbers = listOf(1,2,3,4,5)
var total: Int = 0

for (number in numbers) {
        total+= number
}

println("Summen er: ${total}")
```

```
// Deklarativ eksempel

val numbers = listOf(1,2,3,4,5)
var total: Int = numbers.sum()

println("Summen er: ${total}")
```

Imperativ eller Deklarativ programmering?

```
// Største tal i en liste
val numbers = listOf(3,1,4,1,5,9,2,6,5,3,5)
var maxNumber: Int = numbers.max()


println("Det største tal er: ${maxNumber}")
```

Imperativ eller Deklarativ programmering?

```
// Definer en liste af tal
val numbers = listOf(1, 2, 3, 4, 5, 6, 7, 8, 9, 10)

val evenNumbers = numbers.filter { it % 2 == 0 }

println("Lige tal: $evenNumbers")
```

Imperativ eller Deklarativ programmering?

```
// Største tal i en liste
val numbers = listOf(3, 1, 4, 1, 5, 9, 2, 6, 5, 3, 5)


if (numbers.isNotEmpty()) {
        var maxNumber = numbers[0]
        for (number in numbers) {
                if (number > maxNumber) {
                        maxNumber = number
                }
        }
        println("Det største tal er: $maxNumber")
}
```

Imperativ eller Deklarativ programmering?

```
<!DOCTYPE html>
<html>
    <head>
        <title>Min Webside</title>
    </head>
    <body>
        <h1>Velkommen til min webside</h1>
        <p>Dette er et afsnit med tekst.</p>
        <img src="billede.jpg" alt="Et billede">
    </body>
</html>
```

Reaktiv Programmering

- Fokuseret på at propagere ændringer
- Deklaration af afhængigheder
- Håndtering af asynkrone events
- Håndtering af datastrømme og ændringer i state deklarativt
- Godt til at håndtere mange samtidige interaktioner

Eksempel på imperative programmering

```
var b: Int = 1

val c: Int = 2

var a: Int = b + c

b = 10


println(a)          // prints 3
```

Eksempel på reaktiv programmering

```
var b: Int = 1

val c: Int = 2

var a: Int = b + c

b = 10


println(a)        // prints 12
```

Reaktiv Programmering

- **Stream**: En strøm er en sekvens af data eller begivenheder, der udsendes over tid.
- **Observer**: En observer er en enhed, der udsender data eller begivenheder.
- **Observer**: En observer er en enhed, der modtager data eller begivenheder fra en observer.
- **Operators**: Operatorer er funktioner, der transformerer, filtrerer eller kombinerer strømme.

Reaktiv Programmering

- **Stream**:
- **Observer**:
- **Observer**:
- **Operators**:



Reaktiv Programmering

- **Stream**: postbud
- **Observer**: Afsender
- **Observer**: modtager
- **Operators**: posthuse



Jetpack Compose



- Jetpack Compose er et moderne, deklarativt UI-værktøjssæt til Android.
- Compose gør det nemmere at skrive og vedligeholde din apps brugergrænseflade (UI) ved at tilbyde en deklarativ API, som lader dig rendere din apps UI uden imperativt at skulle ændre i frontend-views.
- Thinking in Compose

https://android-developers.googleblog.com/2020/08/announcing-jetpack-compose-alpha.html

State i Jetpack Compose

- State in Compose er explicit og gives til en composable
- Dette sørger for at decouple UI elementer, og enkapselere state
- Hvis en state ændres opdateres UI elementet automatisk



https://android-developers.googleblog.com/2020/08/announcing-jetpack-compose-alpha.html

State i Jetpack Compose

UI-komponenter (**composables**) fungerer som **observers**, der "observerer" en tilstand (**state**). Når tilstanden ændrer sig, "rekomponeres" de berørte composables automatisk for at afspejle den nye tilstand.



https://android-developers.googleblog.com/2020/08/announcing-jetpack-compose-alpha.html

UI i Jetpack Compose

```
@Composable
private fun AndroidFacts() {
    Column(
        verticalArrangement = Arrangement.Center,
        horizontalAlignment = Alignment.CenterHorizontally
    ) { this: ColumnScope
        Image(
            painter = painterResource(id = R.drawable.android_logo),
            contentDescription = null,
            modifier = Modifier.size(200.dp)
        )

        Spacer(modifier = Modifier.height(10.dp))

        Text(
            text = stringResource(id = R.string.fact),
            color = Color.Black,
            fontSize = 14.sp,
            textAlign = TextAlign.Center,
            modifier = Modifier.padding(bottom = 10.dp)
        )

        Button(
            onClick = { /*TODO*/ }
        ) { this: RowScope
            Text(
                text = "Give me one more fact!",
                color = Color.White
            )
        }
    }
}
```

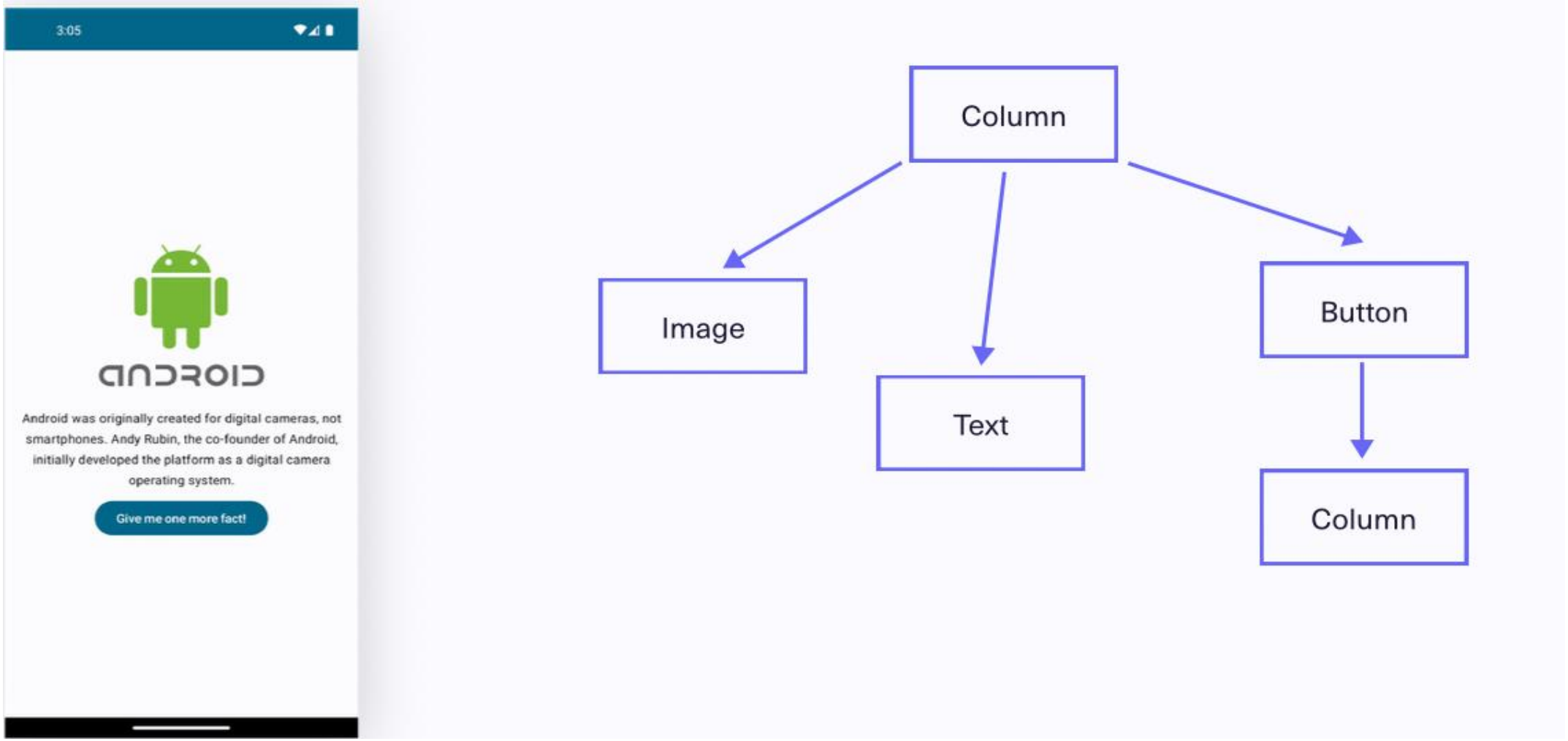


Hver composable gives parametre til at konfigurer dets udseende(**fontSize**, **Color**)

Nogle composables hare ”children composables” (**Column** og **Button**)

UI elementerne danner en træ-struktur, som definer hierarki og informere om hvornår de skal tjekkes for at ”recomposes”

UI træ

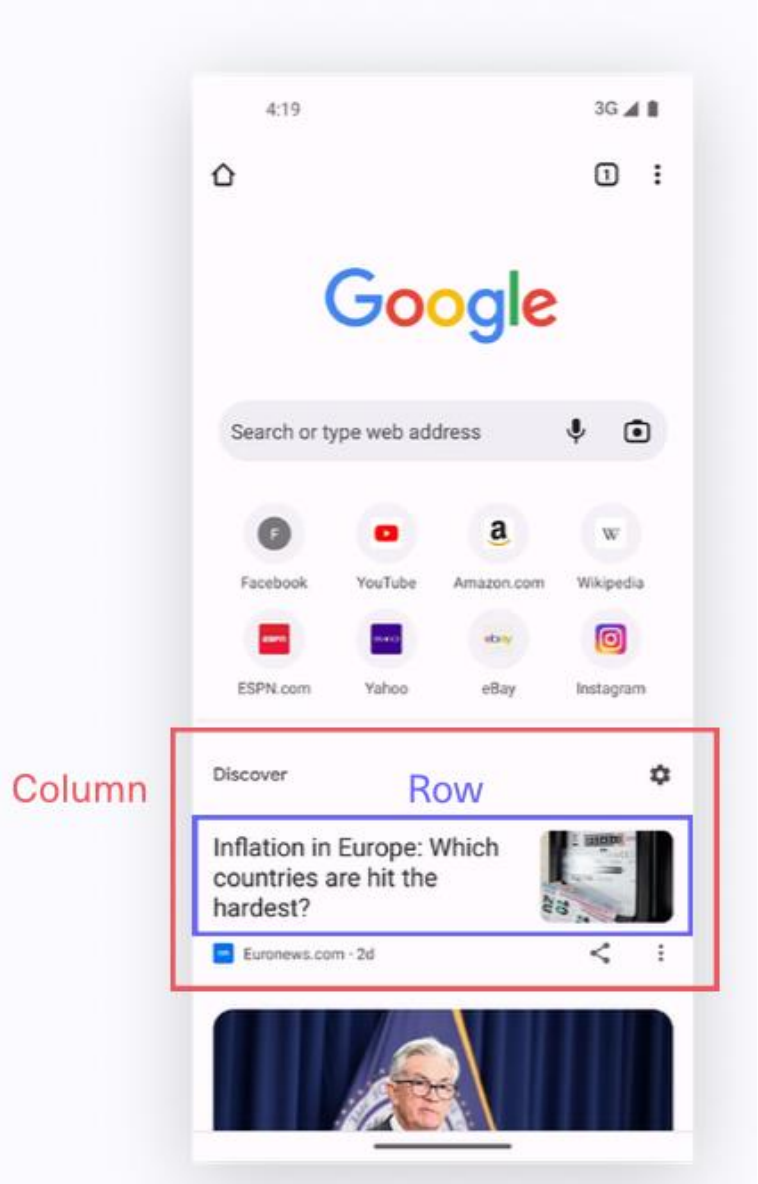

3:05
ANDROID
Android was originally created for digital cameras, not smartphones. Andy Rubin, the co-founder of Android, initially developed the platform as a digital camera operating system.
Give me one more fact!
Column
Image
Text
Button
Column

Layouts

Layouts placeres typisk relativt til hinanden, enten horizontalt eller vertikalt. De kan også plaseres oven på hinanden, som man f.eks. kunne gøre med tekst oven på et billede.

De mest anvendte layouts I Compose til at opnå disse ting er Column, Row, og Box composables, som enten placere deres ”child composables” vertikalt, horizontalt, eller oven på hinanden.

Ud fra desse simple layouts kan man lave mere komplekse layouts og arrangere ens composables.



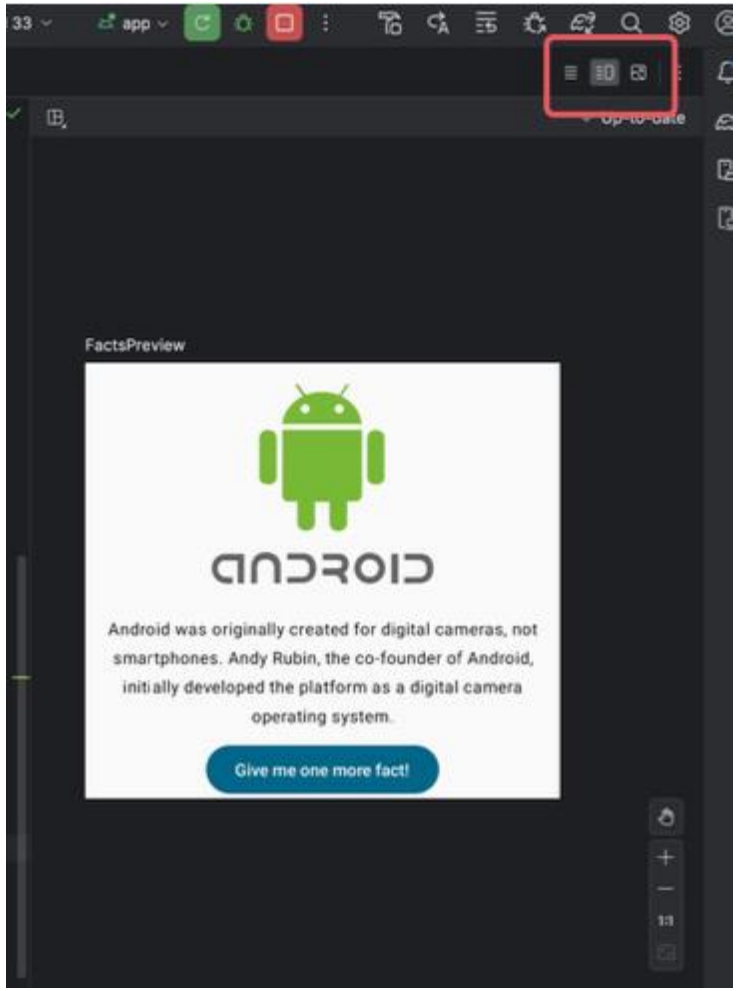
Composable funktioner

```
@Composable
private fun AndroidFacts(fact: String, onButtonClick: () -> Unit) {
    // ...
}
```

- @Composable fortæller compileren at metoden er en composable funktion
- fact og onBottonClick: funktions parameter som konfigurere funktionen
- { … } blokken hvor man kunne kalde andre composable funktioner fra

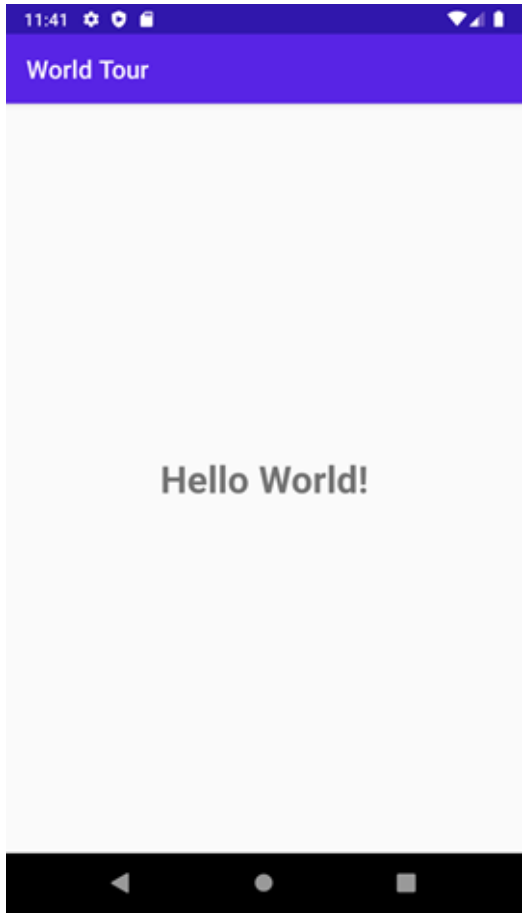
```
@Preview(showBackground = true)
@Composable
private fun FactsPreview() {
    Lecture6Theme {
        AndroidFacts()
    }
}
```

@preview annotationen tillader android studio at lave et preview der løbende opdateres I takt med at man bygger sin UI, uden at skulle bygge og deploy ens app hver gang man har brug for noget visuelt feedback.



Introduktion til mobil applikationer

Hvad er en activity?



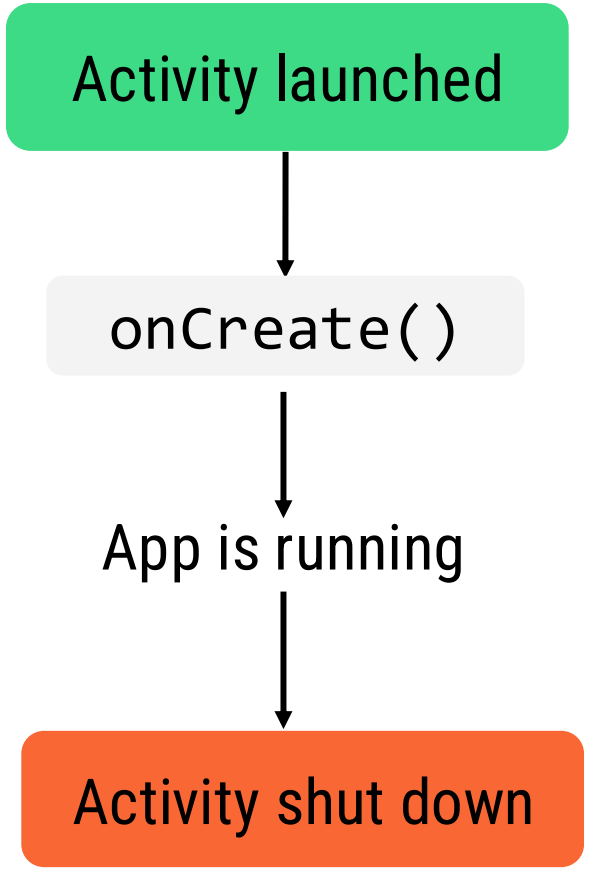
- En Activity er en “måde” for en burger at udføre et mål.
- En mobil app er “composed” af mindst en, og nogle gange flere activities.

MainActivity.kt

```
class MainActivity : AppCompatActivity() {

    override fun onCreate(savedInstanceState: Bundle?) {
        super.onCreate(savedInstanceState)
        setContentView(R.layout.activity_main)
    }
}
```

Activity livscyklus



Implementering af onCreate() callback

Kaldes når mobiltelefonen(operativ systemet) skaber ens activity

```
override fun onCreate(savedInstanceState: Bundle?) {
    super.onCreate(savedInstanceState)
    setContentView(R.layout.activity_main)
}
```

Introduktion til Gradle

Hvad er Gradle?

- Byggeautomatiserings system
- Styrer byggecyklussen gennem en række opgaver (f.eks. kompilerer Kotlin-kildekode, kører tests, installerer appen på enheden)
- Bestemmer den korrekte rækkefølge for de opgaver, der skal køres
- Håndterer afhængigheder mellem projekter og tredjepartsbiblioteker

Gradle build fil

- Holder styr på plugins
- Definer egenskaber for bygge processen
- håndterer afhængigheder
- forbinder til repositorier

Plugins

```
plugins {
    // Android-specifikt plugin
    id("com.android.library")
    // Det vigtigste plugin for KMP
    id("org.jetbrains.kotlin.multiplatform")
    // Til at integrere med iOS via CocoaPods
    id("org.jetbrains.kotlin.native.cocoapods")
    // Plugin til Kotlinx Serialization
    id("org.jetbrains.kotlin.plugin.serialization")
}
```

Platform konfigurationer

```
kotlin {
    // 1. Android Target
    androidTarget {
        // Sætter JVM-versionen for den kompilerede kode
        compilations.all { kotlinOptions.jvmTarget = "1.8” }
    }
}
```

Platform konfigurationer

```
android {
    namespace = "com.eksempel.minapp.shared"
    compileSdk = 34
    defaultConfig {
    minSdk = 24
    }
    compileOptions {
        sourceCompatibility = JavaVersion.VERSION_1_8
        targetCompatibility = JavaVersion.VERSION_1_8
    }
}
```

Afhængigheder

```
sourceSets {
    // 'commonMain' er kildekoden, der deles mellem ALLE platforme
    val commonMain by getting {
        dependencies {
            // Disse biblioteker er tilgængelige i både Android og iOS
            implementation("io.ktor:ktor-client-core:2.3.4")
            implementation("org.jetbrains.kotlinx:kotlinx-coroutines-core:1.7.3")
            implementation("org.jetbrains.kotlinx:kotlinx-serialization-json:1.6.0")
    }
}
```

Afhængigheder

```
        // 'androidMain' er kun for Android-specifik kode
        val androidMain by getting {
            dependencies {
                // Ktor's JVM-motor (OkHttp) er kun nødvendig for Android
                implementation("io.ktor:ktor-client-okhttp:2.3.4")
            }
        }
        // 'iosMain' er kun for iOS-specifik kode
        val iosMain by getting {
            dependencies {
                // Ktor's native iOS-motor er kun nødvendig for iOS
                implementation("io.ktor:ktor-client-darwin:2.3.4")
            }
        }
}
```

Repositorier

```
repositories {
    google() // android-biblioteker fra Google
    mavenCentral() // lager for Java/Kotlin biblioteker
    maven("https://api.ktor.io/m2") // et custom repository
                                    // (for Ktor)
}
```

Gradle håndter også..

- Clean
- Tasks
- InstallDebug

Øvelse

Codelab: Set up Android Studio

Codelab: Build a Basic Layout

- Setting up KMP project
- Shared UI KMP project